

Understanding Log Sources & Investigating with Splunk Module

Introduction To Splunk

Splunk is a scalable, versatile and robust data analytics software solution. Splunk is known for its ability to ingest, index, analyse and visualise huge amounts of machine data. In this regard Splunk can serve as a SIEM for your organisation with a wide array of features. Splunk Enterprise's architecture consists of several layers which can be divided into the following main components:

- Forwarders that are responsible for data collection. Splunk can use different types of forwarders such as:
 - Universal forwarders (UF) that are lightweight agents that collect data and forward it to Splunk indexers without any preprocessing. UFs are usually individual software packages that are easily installed on remote machines without significantly affecting performance.
 - Heavy forwarders (HF) are data forwarders that parse data first, which allows them to route data depending on specific criteria. Because of this heavy forwarders are typically used as dedicated data collection nodes.
 - HTTP event collectors (HECs) directly collect data from applications using token-based JSON or raw API methods. In this case data is sent directly to the indexer level for processing.
- Indexers receive data from the forwarders, organise it and then store it in indexes. While indexing, sets of directories are generated to help categorise the data. Indexers also process search queries from users and return results.
- Search Heads coordinate search jobs by issuing them to the indexers and managing the results. Search heads provide the interface for users to interact with Splunk and can provide supplementary resources to enhance the searching experience. They also allow for the construction of knowledge objects which manipulate data without modifying the original indexed data.
- The deployment server manages the configuration for forwarders, distributing apps and updates.
- The cluster master coordinates the indexers.

- The License master manages the licensing details of the Splunk platform.

Users can interface with the indexers through the search heads either by utilising knowledge objects or by writing queries in SPL (search processing language).

Using SPL

In the following example we will assume that `main` is an index containing Windows security and sysmon logs amongst other data. The most fundamental operation we can perform with SPL is searching. We can expand these search queries with boolean operators as expected:

```
search index="main" "UNKNOWN"
```

In this query we have narrowed down the search to only events stored within the main index and have then used the keyword "UNKNOWN" to search the fields. Note that the search keyword is usually implicit so SPL lets us omit it. For example:

```
index="main" "*UNKNOWN*"
```

Splunk will automatically identify certain data as fields and users can manually define additional fields:

```
index="main" EventCode!=1
```

The fields keyword specifies which fields should be included or excluded from the search results, similar to the select keyword in SQL

```
index="main" sourcetype="WinEventLog:Sysmon" EventCode=1 | fields -user
```

In the above command the fields keyword will exclude the user field from the search results. The table command presents search results in a tabular format:

```
index="main" sourcetype="WinEventLog:Sysmon" EventCode=1 | table _time, host, Image
```

The rename command can be used to rename a field in the search results:

```
index="main" sourcetype="WinEventLog:Sysmon" EventCode=1 | rename Image as Process
```

The dedup command removes duplicate entries:

```
index="main" sourcetype="WinEventLog:Sysmon" EventCode=1 | dedup Image
```

The sort command sorts the search results:

```
index="main" sourcetype="WinEventLog:Sysmon" EventCode=1 | sort - _time
```

In the above example the timestamps are sorted in descending order. The stats command can perform statistical operations. For example if we wanted to write a query which gives us an overview of processes at a given time we could run:

```
index="main" sourcetype="WinEventLog:Sysmon" EventCode=3 | stats count by _time, Image
```

Similarly if we wanted to create a data visualisation of that same data we could use the chart keyword:

```
index="main" sourcetype="WinEventLog:Sysmon" EventCode=3 | chart count by _time, Image
```

The eval command can be used to create or redefine fields. It is comparable to that one command in SQL that I can't remember right now. You use it as follows:

```
index="main" sourcetype="WinEventLog:Sysmon" EventCode=1 | eval Process_Path=lower(Image)
```

It is important to note that eval does not change the underlying data, this additional field only exists for this specific query. If you are masochistic and want to use regex to extract fields you can do that too:

```
index="main" EventCode=4662 | rex max_match=0 "[^%](?<guid>{.*})" | table guid
```

We can enrich our data with external sources by using the lookup command. We have to add our external data as a lookup table in Splunk but once we have done this we can access it using the lookup command. For example, consider a file called malware_lookup.csv that contains a list of filenames and a boolean value to determine that they are or are not malware. Then a command that filters newly spawned processes and compares them against this malware csv file could be as follows:

```
index="main" sourcetype="WinEventLog:Sysmon" EventCode=1 | eval
filename=mvdedup(split(Image, "\\")) | eval filename=mvindex(filename, -1) |
eval filename=lower(filename) | lookup malware_lookup.csv filename OUTPUTNEW
is_malware | table filename, is_malware | dedup filename, is_malware
```

You can also use the inputlookup command to retrieve data from a lookup file without joining it to search results:

```
| inputlookup malware_lookup.csv
```

Every event in Splunk has a timestamp, we can limit searches to specific periods:

```
index="main" earliest=-7d EventCode!=1
```

The transaction command is used in Splunk to group events that share common characteristics. This can be used to track sessions or user activities that span multiple events:

```
index="main" sourcetype="WinEventLog:Sysmon" (EventCode=1 OR EventCode=3) |
transaction Image startswith=eval(EventCode=1) endsuffix=eval(EventCode=3)
maxspan=1m | table Image | dedup Image
```

Splunk supports subsearch which lets you nest searches inside each other. This can be useful to highlight rare or unusual processes:

```
index="main" sourcetype="WinEventLog:Sysmon" EventCode=1 NOT [ search
index="main" sourcetype="WinEventLog:Sysmon" EventCode=1 | top limit=100 Image |
fields Image ] | table _time, Image, CommandLine, User, ComputerName
```

Note that SPL is a vast language with large amounts of flexibility so you should practice writing lots of queries in it to get proficient.

Data Identification

In any SIEM system, understanding the available data sources and the fields within them is crucial. In Splunk we primarily use the Search & Reporting application to do this. Splunk can ingest a wide variety of data sources which we classify into source types that dictate how Splunk formats the data. To identify the available source types after selecting a suitable time range we can run:

```
| eventcount summarize=false index=* | table index
```

```
| metadata type=sourcetypes
```

```
| metadata type=sourcetypes index=* | table sourcetype
```

```
| metadata type=sources index=* | table source
```

Once we know the source types we can investigate what kind of data they contain. Lets say that we are interested in the sourcetype `WinEventLog:Security` . We can use the table command to present the raw data:

```
sourcetype="WinEventLog:Security" | table _raw
```

If we want to see a list of field names only we can use the fieldsummary command:

```
sourcetype="WinEventLog:Security" | fieldsummary
```

Sometimes we might want to see how events are distributed over time. We can use the bucket keyword to group events under some conditions. For example:

```
index=* sourcetype=* | bucket _time span=1d | stats count by _time, index, sourcetype | sort - _time
```

The rare command can help us identify uncommon event types which could be indicative of abnormal behaviour, some examples could be:

```
index=* sourcetype=* | rare limit=10 index, sourcetype
```

```
index="main" | rare limit=20 useother=f ParentImage
```

We can also use the sistats command to explore event diversity:

```
index=* | sistats count by index, sourcetype, source, host
```

There is, however, another way to identify data and fields in Splunk which is by using the User Interface. In Splunk we can navigate to the settings menu and find Data Inputs. Clicking on this will show us a list of data input methods that represents the various sources through which data can be ingested by Splunk. Clicking on these will give us an overview.

If we navigate to the Search & Reporting app we can explore the events in Fast mode, giving us a quick way to scan through our data or we can choose the Verbose mode which lets us dive deeper into the event details. Splunk gives us a selection of Selected fields, which always appear and Interesting fields which appear in at least 20%. We can choose to click All fields to see every field present across all events.

Data Models provide an organised hierarchical view of our data, simplifying datasets in structures. They are designed to make it easier to create reports and dashboards without writing complex SPL queries. To access data models we can go to settings on the Splunk UI and then select Data Models, we can explore these data models in more detail by clicking on any given one of them.

Pivots exist in Splunk and are another extremely powerful way to create complex reports. They operate similarly to pivot tables in Excel.

Using Splunk Applications

Splunk applications are packages we can add to our Splunk deployment to extend its capabilities and manage specific types of data. Each app is tailored to handle data from specific technologies or use cases. Splunk apps enabled the coexistence of multiple workspaces on a single Splunk instance. Splunk apps designed for SIEM purposes provide a range of capabilities to enhance our ability to detect, investigate and respond to threats.

Before deploying such a Splunk app we have to consider the hardware requirements, network traffic and licensing issues. In this module we will look at the Sysmon App for Splunk.

Intrusion Detection With Splunk (Real-world Scenario)

We are already familiar with log exploration on a single machine to pinpoint malicious activity from the previous windows event logs module. We now want to conduct similar investigations but across numerous machines and larger networks. We still need to explore windows event logs but the scope of work will expand significantly. At the start of creating hunts, alerts and queries the volume of data can be intimidating. Part of your job as an analyst is to pinpoint the most meaningful data. To proceed we need data that we can analyse for threat hunting, there are a few different sources we could use for this. One source that Splunk provides is called BOTS, we could also use `nginx_json_logs` which provide dummy logs in JSON format. If you upload data from arbitrary sources, ensure that the source type correctly extracts the JSON by adjusting the Indexed Extractions setting to JSON.

In this module we will focus on a dataset from HTB. Within this data is evidence of different infections. The goal is not to identify every single one but rather to understand how we can detect any sort of attack within such a vast data pool.

Our first objective is to see what we can identify within the Sysmon data, to do this we need to have an understanding of the shape of the data. We will start by listing all of the sourcetypes that exist:

```
index="main" | stats count by sourcetype
```

Now we can query our Sysmon sourcetype and look at the incoming data:

```
index="main" sourcetype="WinEventLog:Sysmon"
```

We can expand one such document and look at what kind of field names might be interesting to search through. We can also use this step to verify we are in fact looking at Sysmon data.

Note that some queries will be much quicker than others so we should aim to make our queries as specific as possible.

```
index="main" ComputerName="*uniwaldo.local"
```

Making progress on our journey, let's pivot our focus towards finding anomalies in our data. We can apply a similar approach to the Windows event log module and look for event codes that we can use to trace potentially malicious activity.

```
index="main" sourcetype="WinEventLog:Sysmon" | stats count by EventCode
```

Remember the Sysmon Event IDs:

Event ID	Event	Desc
1	Process Creation	Look for abnormal parent-child hierarchies.
2	Process changed file creation time	Could indicator "time stomp" attacks but may not always be malicious.
3	Network Connection	Could be useful to find malware making external connections but need other evidence first as this log is huge.
4	Sysmon service state changed	Useful to see if attacks tried to stop sysmon.
5	Process terminated	Probably not as useful as ID 1 but still could contain evidence.
6	Driver loaded	Potential flag for BYOD attacks. Not super common.
7	Image loaded	Allows us to track DLL loads which can be helpful for DLL hijacks.
8	Create Remote thread	Potentially aids in finding injected threads.
10	Process Access	Useful for finding RCE.
11	File Create	Difficult to spot malware in but can be useful for tracing entry points after an investigation.
12	Registry Event (Create/Delete)	Can potentially be malicious. For example, adding malware to startup.
13	Registry Event (Value set)	""
15	File Create Stream Hash	Could help with finding downloaded malware but we will ignore it for now.
16	Sysmon config state change.	Useful for spotting tampering since the config should not change too often and will always be documented when it is changed by IT.
17	Pipe created	Can be useful to look for malware communicated such as with PsExec.

Event ID	Event	Desc
18	Pipe connected	""
22	DNS Event	Probably non-malicious.
23	File Delete	Can be useful for identifying ransomware attacks.
25	Process Tampering	Mini AV alert filter. Useful to pay attention to.

Using these event codes we can run some preliminary queries. For example we can look for unusual parent-child relationships:

```
index="main" sourcetype="WinEventLog:Sysmon" EventCode=1 | stats count by ParentImage, Image
```

This still returns a lot of results so it might be useful to target specific processes we expect to be problematic:

```
index="main" sourcetype="WinEventLog:Sysmon" EventCode=1 (Image="*cmd.exe" OR Image="*powershell.exe") | stats count by ParentImage, Image
```

Quite a few results should stand out to us at this point. In particular we can examine the notepad processes in more detail:

```
index="main" sourcetype="WinEventLog:Sysmon" EventCode=1 (Image="*cmd.exe" OR Image="*powershell.exe") ParentImage="C:\\Windows\\System32\\notepad.exe
```

Here we notice a few things. We can either trace what spawned the notepad process in the first place or we can investigate the mystery IP address that is having a file downloaded from it.

```
index="main" 10.0.0.229 | stats count by sourcetype
```

```
index="main" 10.0.0.229 sourcetype="linux:syslog"
```

The machine itself does not seem to be suspicious however it does alarm us that maybe an internal Linux system has also been compromised.

```
index="main" 10.0.0.229 sourcetype="WinEventLog:sysmon" | stats count by CommandLine, host
```

Due to the suspiciously named powershell files being downloaded we have reason to believe a DCSync attack was launched. We can aim to verify this by running some targeted queries.

```
index="main" EventCode=4662 Access_Mask=0x100 Account_Name!=*$
```

After running this we can explore the documents in more detail and in particular research the GUIDs given to us. Doing this would tell us that most likely the DCSync attack was successful and we have full compromise of our domain. We may now be interested in how the attacker obtained domain admin privileges in the first place. For this we can check for process access to lsass and evaluate if these connections are unusual or expected.

```
index="main" EventCode=10 lsass | stats count by SourceImage
```

Some results should immediately stick out to us:

```
index="main" EventCode=10 lsass SourceImage="C:\\Windows\\System32\\notepad.exe"
```

Sysmon seems to think that this is related to credential dumping and gives us the technique ID so we can explore it further.

Creating Meaningful Alerts

From our investigations of the process that accessed lsass to dump credentials we have ascertained that the API call came from UNKNOWN regions of memory. As of such we want to create an alert to trigger based on similar API calls. We can start by listing all of the call stacks containing UNKNOWN.

```
index="main" CallTrace="*UNKNOWN*" | stats count by EventCode
```

```
index="main" CallTrace="*UNKNOWN*" | stats count by SourceImage
```

```
index="main" CallTrace="*UNKNOWN*" | where SourceImage!=TargetImage | stats count by SourceImage
```

```
index="main" CallTrace="*UNKNOWN*" SourceImage!="*Microsoft.NET*"
CallTrace!="ni.dll" CallTrace!="clr.dll" | where SourceImage!=TargetImage |
stats count by SourceImage
```

In the last query we are excluding anything related to C Sharp due to its JIT compiler which we know will trigger an UNKNOWN memory API call. Note that it is still possible for one of these processes to be malicious but we are being really specific in the malware we are looking for because we want to reduce alert fatigue. Similarly we will remove anything related to WOW64:

```
index="main" CallTrace="*UNKNOWN*" SourceImage!="*Microsoft.NET*"
CallTrace!="ni.dll" CallTrace!="clr.dll" CallTrace!="wow64*" | where
SourceImage!=TargetImage | stats count by SourceImage
```

We can also exclude explorer.exe due to its versatility. Identifying any malicious activity within explorer is incredibly difficult due to the nature of how explorer executes and how many events it triggers.

```
index="main" CallTrace="*UNKNOWN*" SourceImage!="*Microsoft.NET*"
CallTrace!="ni.dll" CallTrace!="clr.dll" CallTrace!="wow64*"
SourceImage!="C:\\Windows\\Explorer.EXE" | where SourceImage!=TargetImage |
stats count by SourceImage
```

Finally we add back in some more useful data to weed out any remaining false positives:

```
index="main" CallTrace="*UNKNOWN*" SourceImage!="*Microsoft.NET*"
CallTrace!="ni.dll" CallTrace!="clr.dll" CallTrace!="wow64*"
SourceImage!="C:\\Windows\\Explorer.EXE" | where SourceImage!=TargetImage |
stats count by SourceImage, TargetImage, CallTrace
```

In a real environment, now that we have the bones of our alert we would need to consider how to make it resilient to bypassing.

Detecting Attacker Behaviour with Splunk

Proficient threat detection is crucial, necessitating a thorough understanding of adversary TTPs. Effective threat detection often revolves around identifying patterns that either match malicious behaviour or deviate significantly from expected behaviour. When crafting queries in SPL we use two main approaches:

- We can use our knowledge of threats and attack vectors to ground our search in terms of known TTPs.
- We rely on mathematical and statistical techniques to highlight anomalies.

These two approaches together give us a comprehensive toolkit for identifying and responding to all sorts of cybersecurity threats. The key in both approaches is to understand our data and

environment and balance the need for accurate detection with the desire to minimise false positives.

SPL Queries from Known TTPs

With this approach we are aiming to recognise patterns that we have seen before which are indicative of specific threats or attack vectors. For example, attackers often leverage native windows binaries such as net.exe to gain insights into the target environment to identify potential privilege escalation opportunities. We can use sysmon event ID 1 to help us find such behaviour

```
index="main" sourcetype="WinEventLog:Sysmon" EventCode=1 Image=*\\ipconfig.exe
OR Image=*\\net.exe OR Image=*\\whoami.exe OR Image=*\\netstat.exe OR
Image=*\\nbtstat.exe OR Image=*\\hostname.exe OR Image=*\\tasklist.exe | stats
count by Image,CommandLine | sort - count
```

Alternatively we could look for network connections to code hosting platforms that might be being used to house payloads. We can use event code 22 to identify accessing these shares:

```
index="main" sourcetype="WinEventLog:Sysmon" EventCode=22 QueryName="*github*"
| stats count by Image, QueryName
```

We could look for the use of tools such as PsExec, which can help attackers execute code as the local system account. We can look for the use of PsExec with event ID 13, 11, 17 or 18.

```
index="main" sourcetype="WinEventLog:Sysmon" EventCode=13
Image="C:\\Windows\\system32\\services.exe"
TargetObject="HKLM\\System\\CurrentControlSet\\Services\\*\\ImagePath" | rex
field=Details "(?<reg_file_name>[^\\" | eval reg_file_name =
lower(reg_file_name), file_name =
if(isnull(file_name),reg_file_name,lower(file_name)) | stats values(Image) AS
Image, values(Details) AS RegistryDetails, values(_time) AS EventTimes, count by
file_name, ComputerName
```

```
index="main" sourcetype="WinEventLog:Sysmon" EventCode=11 Image=System | stats
count by TargetFilename
```

```
index="main" sourcetype="WinEventLog:Sysmon" EventCode=18 Image=System | stats
count by PipeName
```

Attackers may compress tools to transfer them to a compromised host. Whilst this can help evade AV or EDR tools it also leaves a sysmon log of event code 11 when the tools are extracted.

```
index="main" EventCode=11 (TargetFilename="*.zip" OR TargetFilename="*.rar" OR TargetFilename="*.7z") | stats count by ComputerName, User, TargetFilename | sort - count
```

Attackers may transfer tools by using powershell or downloading them through a web browser and we can detect this too:

```
index="main" sourcetype="WinEventLog:Sysmon" EventCode=11 Image="*powershell.exe*" | stats count by Image, TargetFilename | sort + count
```

```
index="main" sourcetype="WinEventLog:Sysmon" EventCode=11 Image="*msedge.exe" TargetFilename="*Zone.Identifier" | stats count by TargetFilename | sort + count
```

The *zone.identifier that we have used in the above query is indicative of a file that has been downloaded from the internet or another potentially untrustworthy source. The Zone.Identifier can contain metadata about where the file was downloaded from. Another thing we could look for is process creation in an unexpected place:

```
index="main" EventCode=1 | regex Image="C:\\\\Users\\\\.*\\\\\\\\Downloads\\\\\\\\.*" | stats count by Image
```

This query checks the users downloads folder which should be fairly easy to correlate to what is normal behaviour (based on time, program executed etc.). We can look for the creation of executables or DLLs created outside of the Windows directory which could be evidence of hijacking:

```
index="main" EventCode=11 (TargetFilename="*.exe" OR TargetFilename="*.dll") TargetFilename!="*\\windows\\*" | stats count by User, TargetFilename | sort + count
```

We could look for suspicious ports under an event code 3

Detecting Attacker Behaviour from Analytics

As previously mentioned, this technique relies heavily on statistical analysis and anomaly detection. We need to profile normal behaviour so we can identify deviations from this baseline.

A good example of this in Splunk is the streamstats command which allows us to perform real-time analysis on data which can be useful for identifying unusual patterns. Consider a scenario where we are monitoring the number of network connections initiated within a certain time frame:

```
index="main" sourcetype="WinEventLog:Sysmon" EventCode=3 | bin _time span=1h |
stats count as NetworkConnections by _time, Image | streamstats time_window=24h
avg(NetworkConnections) as avg stdev(NetworkConnections) as stdev by Image |
eval isOutlier=if(NetworkConnections > (avg + (0.5*stdev)), 1, 0) | search
isOutlier=1
```

This command defines an outlier number of network connections in 24 hours by being 0.5 times the standard deviation away from the mean. We need to be careful when we write such queries as to not freeze by going through too much data but also need write off documents as safe or malicious without more investigation.

Consider trying to write a filter based on command length:

```
index="main" sourcetype="WinEventLog:Sysmon" Image=*cmd.exe | eval
len=len(CommandLine) | table User, len, CommandLine | sort - len
```

We can exclude some of the obviously benign results:

```
index="main" sourcetype="WinEventLog:Sysmon" Image=*cmd.exe
ParentImage!="*msiexec.exe" ParentImage!="*explorer.exe" | eval
len=len(CommandLine) | table User, len, CommandLine | sort - len
```

This should reveal some unusual activity that we have identified because of the statistical analysis that attacker commands are generally longer than regular commands. We could also look at occurrences of cmd prompts being opened:

```
index="main" EventCode=1 (CommandLine="*cmd.exe*") | bucket _time span=1h |
stats count as cmdCount by _time User CommandLine | eventstats avg(cmdCount) as
avg stdev(cmdCount) as stdev | eval isOutlier=if(cmdCount > avg+1.5*stdev, 1, 0)
| search isOutlier=1
```

Some malware might load multiple DLLs in quick succession. We can execute a query like the following to monitor this behaviour:

```
index="main" EventCode=7 | bucket _time span=1h | stats dc(ImageLoaded) as
unique_dlls_loaded by _time, Image | where unique_dlls_loaded > 3 | stats count
by Image, unique_dlls_loaded
```

Then we can filter this data to avoid picking up programs we know are likely to show this behaviour naturally:

```
index="main" EventCode=7 NOT (Image="C:\\Windows\\System32*") NOT  
(Image="C:\\Program Files (x86)*") NOT (Image="C:\\Program Files*") NOT  
(Image="C:\\ProgramData*") NOT (Image="C:\\Users\\waldo\\AppData*") | bucket  
_time span=1h | stats dc(ImageLoaded) as unique_dlls_loaded by _time, Image |  
where unique_dlls_loaded > 3 | stats count by Image, unique_dlls_loaded | sort -  
unique_dlls_loaded
```

We can make use of the transaction keyword to look for the same process being executed on the same computer multiple times which would then need further investigation to work out if it was benign or malicious:

```
index="main" sourcetype="WinEventLog:Sysmon" EventCode=1 | transaction  
ComputerName, Image | where mvcount(ProcessGuid) > 1 | stats count by Image,  
ParentImage
```

```
index="main" sourcetype="WinEventLog:Sysmon" EventCode=1 | transaction  
ComputerName, Image | where mvcount(ProcessGuid) > 1 | search  
Image="C:\\Windows\\System32\\rundll32.exe"  
ParentImage="C:\\Windows\\System32\\svchost.exe" | table CommandLine,  
ParentCommandLine
```